JUnit Basic Testing Exercises

Exercise 1: Setting Up JUnit

Source Code

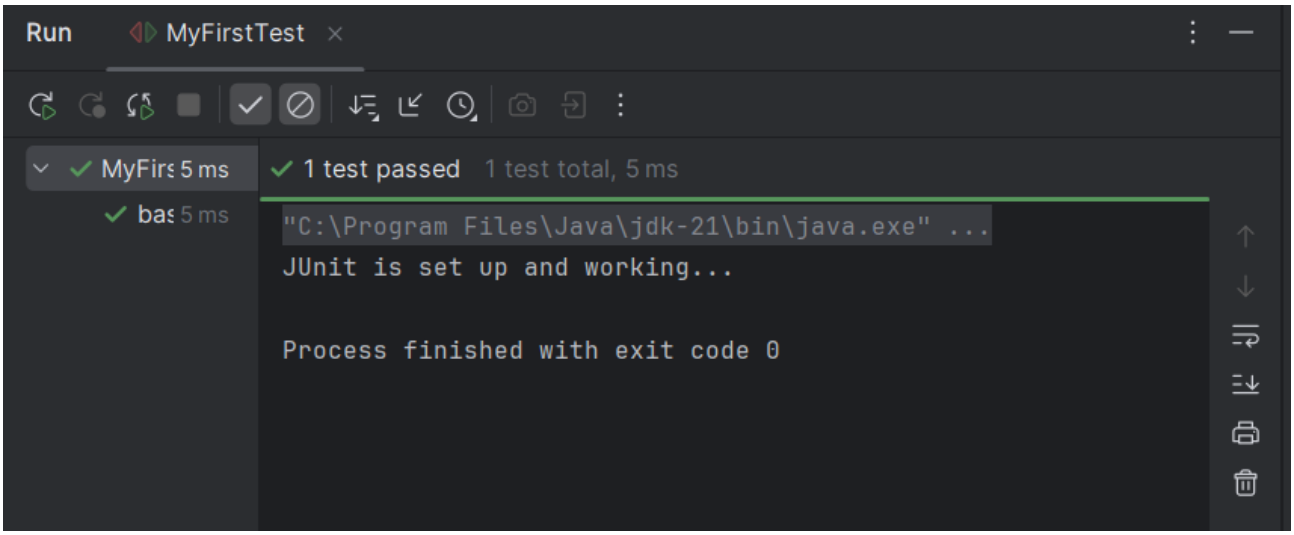
pom.xml - Libraries and Dependencies

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <groupId>com.example</groupId>
    <artifactId>JUnitSetup</artifactId>
    <version>1.0-SNAPSHOT</version>

    <dependencies>
        <dependency>
            <groupId>junit</groupId>
            <artifactId>junit</artifactId>
            <version>4.13.2</version>
            <scope>test</scope>
        </dependency>
    </dependencies>
</project>
```

MyFirstTest.java

```
package com.example;

import org.junit.Test;

public class MyFirstTest {

    @Test
    public void basicTest() {
        System.out.println("JUnit is set up and working...");
    }
}
```

Output Evidence



Output Screenshot

Exercise 2: Basic JUnit Test

Source Code

pom.xml - Libraries and Dependencies

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
```

```
         https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <groupId>com.example</groupId>
    <artifactId>JUnitSetup</artifactId>
    <version>1.0-SNAPSHOT</version>

    <dependencies>
        <dependency>
            <groupId>junit</groupId>
            <artifactId>junit</artifactId>
            <version>4.13.2</version>
            <scope>test</scope>
        </dependency>
    </dependencies>
</project>
```

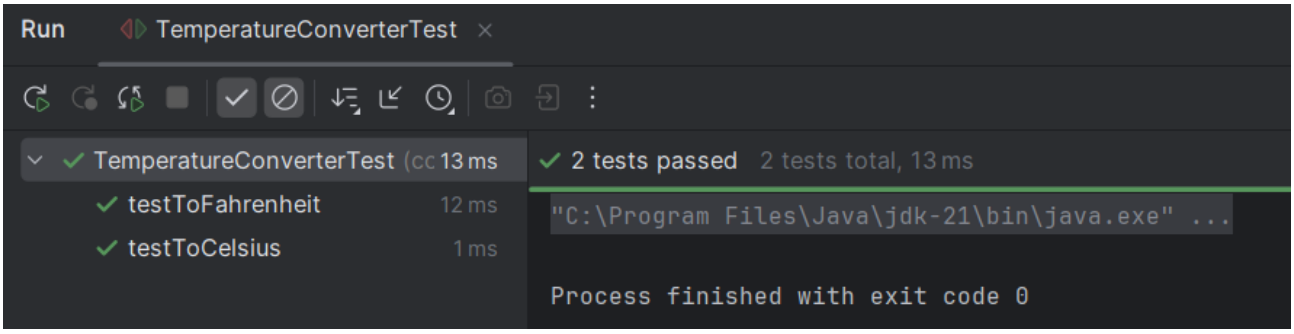
TemperatureConverter.java

```
package com.example;

public class TemperatureConverter {

    public double toFahrenheit(double celsius) {
        return (celsius * 9 / 5) + 32;
    }

    public double toCelsius(double fahrenheit) {
        return (fahrenheit - 32) * 5 / 9;
    }
}
```

TemperatureConverterTest.java

```
package com.example;

import org.junit.Test;
import static org.junit.Assert.*;

public class TemperatureConverterTest {

    TemperatureConverter converter = new TemperatureConverter();

    @Test
    public void testToFahrenheit() {
        double result = converter.toFahrenheit(0);
        assertEquals(32.0, result, 0.001);
    }

    @Test
    public void testToCelsius() {
        double result = converter.toCelsius(212);
        assertEquals(100.0, result, 0.001);
    }
}
```

Output Evidence



Output Screenshot

Exercise 3: Assertions In JUnit

Source Code

pom.xml - Libraries and Dependencies

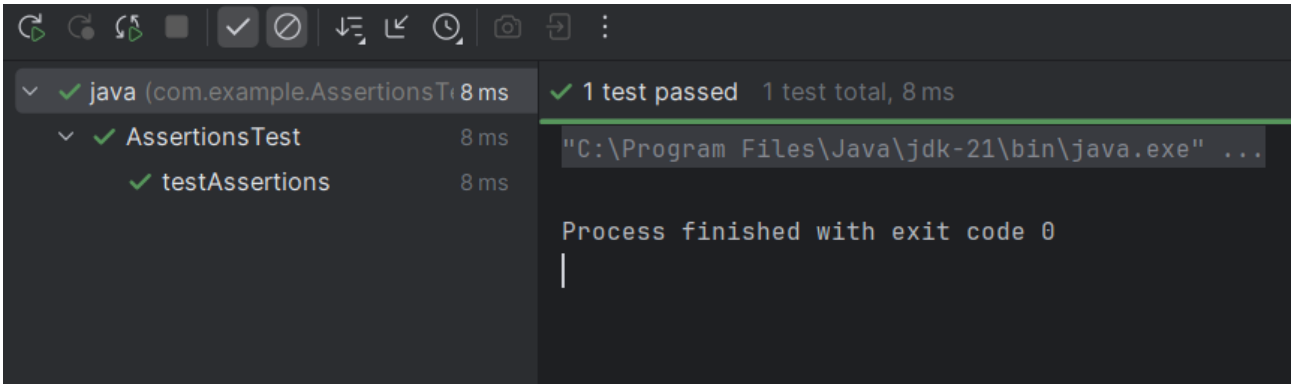
```
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
```

```
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <groupId>com.example</groupId>
    <artifactId>JUnitSetup</artifactId>
    <version>1.0-SNAPSHOT</version>

    <dependencies>
        <dependency>
            <groupId>junit</groupId>
            <artifactId>junit</artifactId>
            <version>4.13.2</version>
            <scope>test</scope>
        </dependency>
    </dependencies>
</project>
```

AssertionsTest.java

```
package com.example.AssertionsTest.java;

import org.junit.Test;
import static org.junit.Assert.*;

public class AssertionsTest {

    @Test
    public void testAssertions() {
        // Assert equals
        assertEquals(5, 2 + 3);

        // Assert true
        assertTrue(5 > 3);

        // Assert false
        assertFalse(5 < 3);

        // Assert null
        assertNull(null);

        // Assert not null
        assertNotNull(new Object());
    }
}
```

Output Evidence



Output Screenshot

Exercise 4: Arrange Act Assert Pattern

Source Code

pom.xml - Libraries and Dependencies

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <groupId>com.example</groupId>
    <artifactId>JUnitSetup</artifactId>
    <version>1.0-SNAPSHOT</version>
```

```
    <dependencies>
        <dependency>
            <groupId>junit</groupId>
            <artifactId>junit</artifactId>
            <version>4.13.2</version>
            <scope>test</scope>
        </dependency>
    </dependencies>
</project>
```

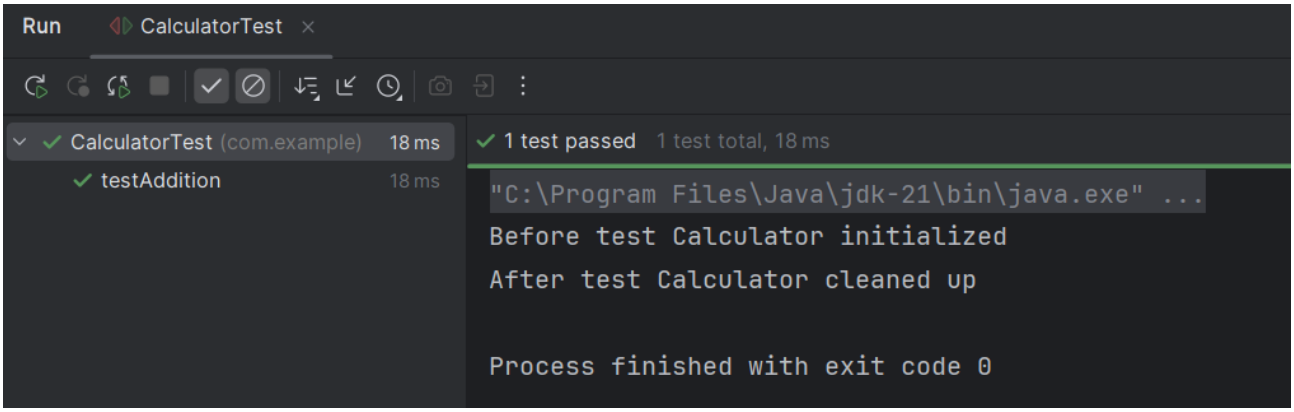
Calculator.java

```
package com.example;

public class Calculator {
    public int add(int a, int b) {
        return a + b;
    }
}
```

CalculatorTest.java

```
package com.example;

import org.junit.Before;
import org.junit.After;
import org.junit.Test;
import static org.junit.Assert.*;

public class CalculatorTest {

    private Calculator calculator;

    // Arrange setup method
    @Before
    public void setUp() {
        calculator = new Calculator();
        System.out.println("Before test Calculator initialized");
    }

    // Actual test method
    @Test
    public void testAddition() {

        // Act
        int result = calculator.add(2, 3);

        // Assert
        assertEquals(5, result);
    }

    // Teardown cleanup method
    @After
    public void tearDown() {
        calculator = null;
        System.out.println("After test Calculator cleaned up");
    }
}
```

Output Evidence



Output Screenshot

Mockito Exercises

Exercise 1: Mocking And Stubbing

Source Code

pom.xml - Libraries and Dependencies

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
             https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <groupId>com.example</groupId>
    <artifactId>MockitoExercise</artifactId>
    <version>1.0-SNAPSHOT</version>

    <dependencies>
        <dependency>
            <groupId>junit</groupId>
            <artifactId>junit</artifactId>
            <version>4.13.2</version>
            <scope>test</scope>
        </dependency>

        <!-- Mockito -->
        <dependency>
            <groupId>org.mockito</groupId>
            <artifactId>mockito-core</artifactId>
            <version>5.10.0</version>
            <scope>test</scope>
        </dependency>
    </dependencies>
</project>
```

ExternalApi.java

```
package com.example;

public interface ExternalApi {
    String getData();
}
```

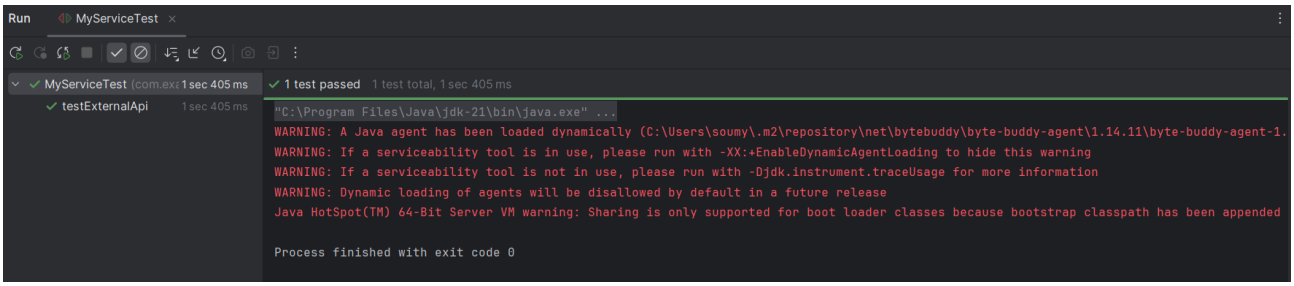
MyService.java

```
package com.example;

public class MyService {
    private final ExternalApi api;

    public MyService(ExternalApi api) {
        this.api = api;
    }

    public String fetchData() {
        return api.getData();
    }
}
```

MyServiceTest.java

```
package com.example;

import org.junit.Test;
import static org.mockito.Mockito.*;
import static org.junit.Assert.*;

public class MyServiceTest {

    @Test
    public void testExternalApi() {
        // Mock of the ExternalApi
        ExternalApi mockApi = mock(ExternalApi.class);

        // Stub the getData() method
        when(mockApi.getData()).thenReturn("Mock Data");

        MyService service = new MyService(mockApi);

        String result = service.fetchData();
        assertEquals("Mock Data", result);
    }
}
```

Output Evidence



Output Screenshot

Exercise 2: Verifying Interactions

Source Code

pom.xml - Libraries and Dependencies

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         https://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>
    <groupId>com.example</groupId>
    <artifactId>MockitoVerifyInteraction</artifactId>
    <version>1.0-SNAPSHOT</version>

    <dependencies>
        <dependency>
            <groupId>junit</groupId>
            <artifactId>junit</artifactId>
            <version>4.13.2</version>
            <scope>test</scope>
        </dependency>


        <dependency>
            <groupId>org.mockito</groupId>
            <artifactId>mockito-core</artifactId>
            <version>5.10.0</version>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>org.mockito</groupId>
            <artifactId>mockito-core</artifactId>
            <version>5.10.0</version>
            <scope>test</scope>
        </dependency>
    </dependencies>
</project>
```

ExternalApi.java

```
package com.example;

public interface ExternalApi {
    String getData();
}
```

MyService.java

```
package com.example;

public class MyService {

    private final ExternalApi api;

    public MyService(ExternalApi api) {
        this.api = api;
    }

    public String fetchData() {
        return api.getData();
    }
}
```

MyServiceTest.java

```
package com.example;

import org.junit.Test;
import org.mockito.Mockito;

import static org.mockito.Mockito.*;

public class MyServiceTest {

    @Test
    public void testVerifyInteraction() {
        // Create mock
        ExternalApi mockApi = Mockito.mock(ExternalApi.class);

        //Use mock in service
        MyService service = new MyService(mockApi);
        service.fetchData();

        // Verify interaction
        verify(mockApi).getData();
    }
}
```

Output Evidence

Output Screenshot

Exercise 3: Argument Matching

Source Code

pom.xml - Libraries and Dependencies

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         https://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>
    <groupId>com.example</groupId>
    <artifactId>MockitoArgumentMatching</artifactId>
    <version>1.0-SNAPSHOT</version>

    <dependencies>
        <dependency>
            <groupId>junit</groupId>
            <artifactId>junit</artifactId>
            <version>4.13.2</version>
            <scope>test</scope>
        </dependency>

        <dependency>
            <groupId>org.mockito</groupId>
            <artifactId>mockito-core</artifactId>
            <version>5.10.0</version>
            <scope>test</scope>
        </dependency>
    </dependencies>
</project>
```

AlertService.java

```
package com.example;

public class AlertService {

    private final Notifier notifier;
```

```
    public AlertService(Notifier notifier) {
        this.notifier = notifier;
    }

    public void sendAlert(String userId) {
        String msg = "Your session is about to expire!";
        notifier.send(userId, msg);
    }
}
```

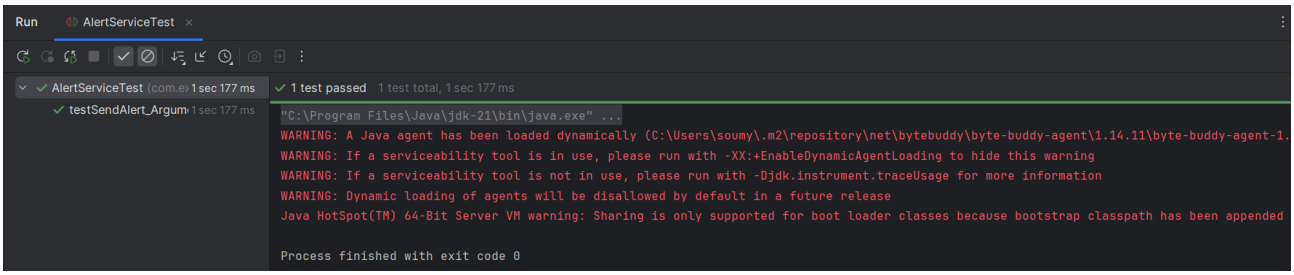
Notifier.java

```
package com.example;

public interface Notifier {
    void send(String userId, String message);
}
```

AlertServiceTest.java

```
package com.example;

import org.junit.Test;
import static org.mockito.Mockito.*;
import static org.mockito.ArgumentMatchers.*;

public class AlertServiceTest {

    @Test
    public void testSendAlert_ArgumentMatching() {

        Notifier mockNotifier = mock(Notifier.class);
        AlertService service = new AlertService(mockNotifier);

        // Act
        service.sendAlert("user_101");

        // Verify using argument matchers
        verify(mockNotifier).send(eq("user_101"), contains("expire"));
    }
}
```

Output Evidence



Output Screenshot

Exercise 4: Handling Void Methods

Source Code

pom.xml - Libraries and Dependencies

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         https://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>
    <groupId>com.example</groupId>
    <artifactId>MockitoArgumentMatching</artifactId>
    <version>1.0-SNAPSHOT</version>

    <dependencies>
        <dependency>
            <groupId>junit</groupId>
            <artifactId>junit</artifactId>
            <version>4.13.2</version>
            <scope>test</scope>
        </dependency>
```

```
        <dependency>
            <groupId>org.mockito</groupId>
            <artifactId>mockito-core</artifactId>
            <version>5.10.0</version>
            <scope>test</scope>
        </dependency>
    </dependencies>
</project>
```

EmailService.java

```
package com.example;

public interface EmailService {
    void sendEmail(String to, String subject, String body);
}
```

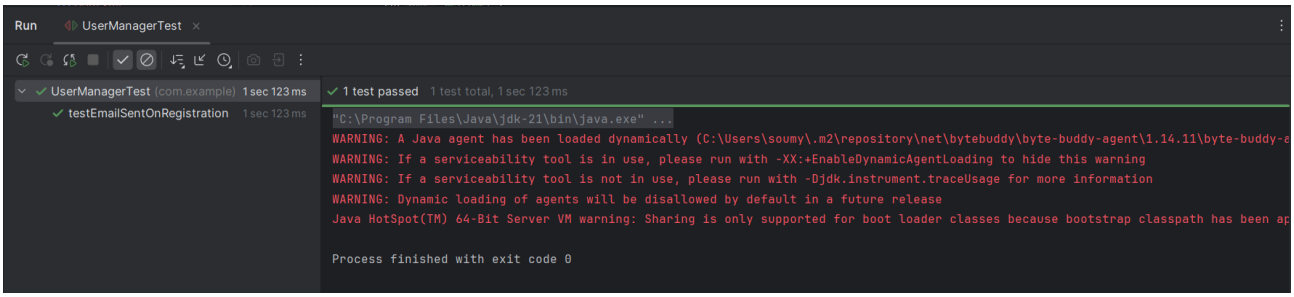
UserManager.java

```
package com.example;

public class UserManager {

    private final EmailService emailService;

    public UserManager(EmailService emailService) {
        this.emailService = emailService;
    }

    public void registerUser(String email) {
        String subject = "Welcome!";
        String body = "Thanks for registering.";
        emailService.sendEmail(email, subject, body);
    }
}
```

UserManagerTest.java

```
package com.example;

import org.junit.Test;
import static org.mockito.Mockito.*;

public class UserManagerTest {

    @Test
    public void testEmailSentOnRegistration() {
        EmailService mockEmailService = mock(EmailService.class);
        UserManager manager = new UserManager(mockEmailService);

        // Call the method under test
        manager.registerUser("test@example.com");

        // Verify the void method was called
        verify(mockEmailService).sendEmail(
                eq("test@example.com"),
                eq("Welcome!"),
                eq("Thanks for registering.")
        );
    }
}
```

Output Evidence



Output Screenshot

Exercise 5: Multiple Returns

Source Code

pom.xml - Libraries and Dependencies

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         https://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>
    <groupId>com.example</groupId>
    <artifactId>MockitoArgumentMatching</artifactId>
    <version>1.0-SNAPSHOT</version>

    <dependencies>
        <dependency>
            <groupId>junit</groupId>
            <artifactId>junit</artifactId>
            <version>4.13.2</version>
            <scope>test</scope>
        </dependency>

        <dependency>
            <groupId>org.mockito</groupId>
            <artifactId>mockito-core</artifactId>
            <version>5.10.0</version>
            <scope>test</scope>
        </dependency>
    </dependencies>
</project>
```

WeatherApi.java

```
package com.example;

public interface WeatherApi {
    String getForecast();
}
```

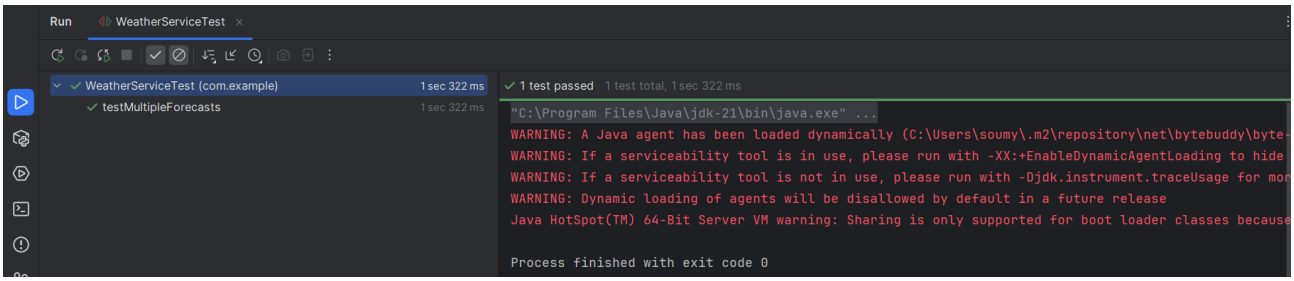
WeatherService.java

```
package com.example;

public class WeatherService {

    private final WeatherApi api;

    public WeatherService(WeatherApi api) {
        this.api = api;
    }

    public String[] fetchForecasts() {
        return new String[] {
                api.getForecast(),
                api.getForecast(),
                api.getForecast()
        };
    }
}
```

WeatherServiceTest.java

```
package com.example;

import org.junit.Test;
import static org.junit.Assert.*;
import static org.mockito.Mockito.*;

public class WeatherServiceTest {

    @Test
    public void testMultipleForecasts() {

        WeatherApi mockApi = mock(WeatherApi.class);

        when(mockApi.getForecast())
                .thenReturn("Sunny")
                .thenReturn("Cloudy")
                .thenReturn("Rainy");

        WeatherService service = new WeatherService(mockApi);
        String[] result = service.fetchForecasts();

        //Verify the results
        assertArrayEquals(new String[] { "Sunny", "Cloudy", "Rainy" }, result);
    }
}
```

Output Evidence



Output Screenshot

PLSQL Exercises

Shared Reference Files

Source Code

Sample_Tables_Data.txt

```
CREATE TABLE Customers (
    CustomerID NUMBER PRIMARY KEY,
    Name VARCHAR2(100),
    DOB DATE,
    Balance NUMBER,
    LastModified DATE
);

CREATE TABLE Accounts (
    AccountID NUMBER PRIMARY KEY,
    CustomerID NUMBER,
    AccountType VARCHAR2(20),
    Balance NUMBER,
    LastModified DATE,
    FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)
);

CREATE TABLE Transactions (
    TransactionID NUMBER PRIMARY KEY,
    AccountID NUMBER,
    TransactionDate DATE,
    Amount NUMBER,
    TransactionType VARCHAR2(10),
    FOREIGN KEY (AccountID) REFERENCES Accounts(AccountID)
);

CREATE TABLE Loans (
    LoanID NUMBER PRIMARY KEY,
    CustomerID NUMBER,
    LoanAmount NUMBER,
    InterestRate NUMBER,
    StartDate DATE,
    EndDate DATE,
    FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)
);

CREATE TABLE Employees (
    EmployeeID NUMBER PRIMARY KEY,
    Name VARCHAR2(100),
    Position VARCHAR2(50),
    Salary NUMBER,
    Department VARCHAR2(50),
    HireDate DATE
);

-- Sample Data Insertion

INSERT INTO Customers (CustomerID, Name, DOB, Balance, LastModified)
VALUES (1, 'John Doe', TO_DATE('1985-05-15', 'YYYY-MM-DD'), 1000, SYSDATE);

INSERT INTO Customers (CustomerID, Name, DOB, Balance, LastModified)
VALUES (2, 'Jane Smith', TO_DATE('1990-07-20', 'YYYY-MM-DD'), 1500, SYSDATE);

INSERT INTO Accounts (AccountID, CustomerID, AccountType, Balance, LastModified)
```

Sample_Tables_Data.txt (continued)

```
VALUES (1, 1, 'Savings', 1000, SYSDATE);

INSERT INTO Accounts (AccountID, CustomerID, AccountType, Balance, LastModified)
VALUES (2, 2, 'Checking', 1500, SYSDATE);

INSERT INTO Transactions (TransactionID, AccountID, TransactionDate, Amount, TransactionType)
VALUES (1, 1, SYSDATE, 200, 'Deposit');

INSERT INTO Transactions (TransactionID, AccountID, TransactionDate, Amount, TransactionType)
VALUES (2, 2, SYSDATE, 300, 'Withdrawal');

INSERT INTO Loans (LoanID, CustomerID, LoanAmount, InterestRate, StartDate, EndDate)
VALUES (1, 1, 5000, 5, SYSDATE, ADD_MONTHS(SYSDATE, 60));

INSERT INTO Employees (EmployeeID, Name, Position, Salary, Department, HireDate)
VALUES (1, 'Alice Johnson', 'Manager', 70000, 'HR', TO_DATE('2015-06-15', 'YYYY-MM-DD'));

INSERT INTO Employees (EmployeeID, Name, Position, Salary, Department, HireDate)
```

```
VALUES (2, 'Bob Brown', 'Developer', 60000, 'IT', TO_DATE('2017-03-20', 'YYYY-MM-DD'));
```

Output Evidence

Output screenshot is not available for this exercise.

Exercise 1: Control Structures

Source Code

scenario1_discount_age.sql

```
BEGIN
   FOR c IN (SELECT customer_id, DOB FROM Customers) LOOP
      IF MONTHS_BETWEEN(SYSDATE, c.DOB)/12 > 60 THEN
         UPDATE Loans
         SET InterestRate = InterestRate - 1
         WHERE CustomerID = c.customer_id;

         DBMS_OUTPUT.PUT_LINE('Discount applied for Customer ID: ' || c.customer_id);
      END IF;
   END LOOP;
   COMMIT;
END;
/
```

scenario2_vip_status_balance.sql

```
ALTER TABLE Customers ADD IsVIP VARCHAR2(5);

BEGIN
   FOR c IN (SELECT CustomerID, Balance FROM Customers) LOOP
      IF c.Balance > 10000 THEN
         UPDATE Customers
         SET IsVIP = 'TRUE'
         WHERE CustomerID = c.CustomerID;

         DBMS_OUTPUT.PUT_LINE('Customer ID ' || c.CustomerID || ' set as VIP.');
      END IF;
   END LOOP;
   COMMIT;
END;
/
```

scenario3_loan_due_reminder.sql

```
BEGIN
   FOR l IN (
      SELECT LoanID, CustomerID, EndDate
      FROM Loans
      WHERE EndDate BETWEEN SYSDATE AND SYSDATE + 30
   ) LOOP
      DBMS_OUTPUT.PUT_LINE('Reminder: Loan ID ' || l.LoanID ||
                           ' for Customer ID ' || l.CustomerID ||
                           ' is due on ' || TO_CHAR(l.EndDate, 'YYYY-MM-DD'));
   END LOOP;
END;
/
```

Output Evidence

Output screenshot is not available for this exercise.

Exercise 2: Error Handling

Source Code

scenario1_safe_transfer_funds.sql

```
CREATE OR REPLACE PROCEDURE SafeTransferFunds (
   from_acc_id IN NUMBER,
   to_acc_id IN NUMBER,
   amount IN NUMBER
) IS
   from_balance NUMBER;
BEGIN
   SELECT Balance INTO from_balance FROM Accounts WHERE AccountID = from_acc_id;
```

```
    IF from_balance < amount THEN
       RAISE_APPLICATION_ERROR(-20001, 'Insufficient funds');
    END IF;

     -- Subtract from sender
    UPDATE Accounts SET Balance = Balance - amount, LastModified = SYSDATE WHERE AccountID =
        from_acc_id;

     -- Add to receiver
    UPDATE Accounts SET Balance = Balance + amount, LastModified = SYSDATE WHERE AccountID =
        to_acc_id;

    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Transfer successful from Account ' || from_acc_id || ' to Account ' ||
        to_acc_id);

EXCEPTION
   WHEN OTHERS THEN
      ROLLBACK;
      DBMS_OUTPUT.PUT_LINE('Error in SafeTransferFunds: ' || SQLERRM);
END;
/
```

scenario2_update_salary.sql

```
CREATE OR REPLACE PROCEDURE UpdateSalary (
    emp_id IN NUMBER,
    percent IN NUMBER
)
AS
    current_salary NUMBER;
BEGIN
    -- Get current salary
    SELECT Salary INTO current_salary FROM Employees WHERE EmployeeID = emp_id;

    -- Update salary with bonus
    UPDATE Employees
    SET Salary = current_salary + (current_salary * percent / 100)
    WHERE EmployeeID = emp_id;

    COMMIT;

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        INSERT INTO ErrorLog (message, log_date)
        VALUES ('Employee ID ' || emp_id || ' not found.', SYSDATE);
    WHEN OTHERS THEN
        INSERT INTO ErrorLog (message, log_date)
        VALUES ('Error in UpdateSalary for ID ' || emp_id || ': ' || SQLERRM, SYSDATE);
        ROLLBACK;
END;
/
```

scenario3_add_new_customer.sql

```
CREATE OR REPLACE PROCEDURE AddNewCustomer (
    cid IN NUMBER,
    cname IN VARCHAR2,
    cdob IN DATE,
    cbalance IN NUMBER
)
AS
BEGIN
    -- Insert new customer
    INSERT INTO Customers (CustomerID, Name, DOB, Balance, LastModified)
    VALUES (cid, cname, cdob, cbalance, SYSDATE);

    COMMIT;

EXCEPTION
    WHEN DUP_VAL_ON_INDEX THEN
        INSERT INTO ErrorLog (message, log_date)
        VALUES ('Customer ID ' || cid || ' already exists.', SYSDATE);
    WHEN OTHERS THEN
        INSERT INTO ErrorLog (message, log_date)
        VALUES ('Error in AddNewCustomer for ID ' || cid || ': ' || SQLERRM, SYSDATE);
        ROLLBACK;
END;
/
```

Output Evidence

Output screenshot is not available for this exercise.

Exercise 3: Stored Procedures

Source Code

scenario1_process_monthly_interest.sql

```
CREATE OR REPLACE PROCEDURE ProcessMonthlyInterest IS
BEGIN
   FOR acc IN (SELECT AccountID, Balance FROM Accounts WHERE AccountType = 'Savings') LOOP
      UPDATE Accounts
      SET Balance = acc.Balance + (acc.Balance * 0.01),
          LastModified = SYSDATE
      WHERE AccountID = acc.AccountID;

      DBMS_OUTPUT.PUT_LINE('Interest applied to Account ID: ' || acc.AccountID);
   END LOOP;

   COMMIT;
END;
/
```

scenario2_update_employee_bonus.sql

```
CREATE OR REPLACE PROCEDURE UpdateEmployeeBonus (
   p_department IN VARCHAR2,
   p_bonus_pct IN NUMBER
) IS
BEGIN
   FOR emp IN (SELECT EmployeeID, Salary FROM Employees WHERE Department = p_department) LOOP
      UPDATE Employees
      SET Salary = emp.Salary + (emp.Salary * p_bonus_pct / 100)
      WHERE EmployeeID = emp.EmployeeID;

      DBMS_OUTPUT.PUT_LINE('Bonus applied to Employee ID: ' || emp.EmployeeID);
   END LOOP;

   COMMIT;
END;
/
```

scenario3_transfer_funds.sql

```
CREATE OR REPLACE PROCEDURE TransferFunds (
   p_from_account IN NUMBER,
   p_to_account IN NUMBER,
   p_amount IN NUMBER
) IS
   v_balance NUMBER;
BEGIN
   -- Get balance of source acc
   SELECT Balance INTO v_balance FROM Accounts WHERE AccountID = p_from_account;

   IF v_balance >= p_amount THEN
      -- Deduct from source
      UPDATE Accounts
      SET Balance = Balance - p_amount,
          LastModified = SYSDATE
      WHERE AccountID = p_from_account;

      -- Add to destination
      UPDATE Accounts
      SET Balance = Balance + p_amount,
          LastModified = SYSDATE
      WHERE AccountID = p_to_account;

      DBMS_OUTPUT.PUT_LINE('Transferred ' || p_amount || ' from Account ' || p_from_account || '
           to Account ' || p_to_account);
   ELSE
      DBMS_OUTPUT.PUT_LINE('Insufficient balance in Account ' || p_from_account);
   END IF;

   COMMIT;
END;
/
```

Output Evidence

Output screenshot is not available for this exercise.

Exercise 4: Functions

Source Code

scenario1_calculate_age.sql

```
CREATE OR REPLACE FUNCTION CalculateAge (
    dob IN DATE
) RETURN NUMBER
IS
    age NUMBER;
BEGIN
    age := FLOOR(MONTHS_BETWEEN(SYSDATE, dob) / 12);
    RETURN age;
END;
/
```

scenario2_calculate_monthly_installment.sql

```
CREATE OR REPLACE FUNCTION CalculateMonthlyInstallment (
    loan_amount IN NUMBER,
    interest_rate IN NUMBER,
    duration_years IN NUMBER
) RETURN NUMBER
IS
    monthly_installment NUMBER;
BEGIN
    monthly_installment := (loan_amount + (loan_amount * interest_rate * duration_years / 100))
        / (duration_years * 12);
    RETURN monthly_installment;
END;
/
```

scenario3_has_sufficient_balance.sql

```
CREATE OR REPLACE FUNCTION HasSufficientBalance (
    acc_id IN NUMBER,
    amount IN NUMBER
) RETURN BOOLEAN
IS
    acc_balance NUMBER;
BEGIN
    SELECT Balance INTO acc_balance FROM Accounts WHERE AccountID = acc_id;

    IF acc_balance >= amount THEN
        RETURN TRUE;
    ELSE
        RETURN FALSE;
    END IF;

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RETURN FALSE;
END;
/



//Test Block
DECLARE
    result BOOLEAN;
BEGIN
    result := HasSufficientBalance(1, 500);
    IF result THEN
        DBMS_OUTPUT.PUT_LINE('Sufficient balance.');
    ELSE
        DBMS_OUTPUT.PUT_LINE('Insufficient balance.');
    END IF;
END;
/
```

Output Evidence

Output screenshot is not available for this exercise.

Exercise 5: Triggers

Source Code

scenario1_update_customer_last_modified.sql

```
CREATE OR REPLACE TRIGGER UpdateCustomerLastModified
BEFORE UPDATE ON Customers
FOR EACH ROW
```

```
BEGIN
    :NEW.LastModified := SYSDATE;
END;
/
```

scenario2_log_transaction.sql

```
CREATE OR REPLACE TRIGGER LogTransaction
AFTER INSERT ON Transactions
FOR EACH ROW
BEGIN
    INSERT INTO AuditLog (TransactionID, AccountID, Amount, TransactionType, LogDate)
    VALUES (:NEW.TransactionID, :NEW.AccountID, :NEW.Amount, :NEW.TransactionType, SYSDATE);
END;
/



//Track log details
CREATE TABLE AuditLog (
    LogID NUMBER GENERATED BY DEFAULT ON NULL AS IDENTITY PRIMARY KEY,
    TransactionID NUMBER,
    AccountID NUMBER,
    Amount NUMBER,
    TransactionType VARCHAR2(10),
    LogDate DATE
);
```

scenario3_check_transaction_rules.sql

```
CREATE OR REPLACE TRIGGER CheckTransactionRules
BEFORE INSERT ON Transactions
FOR EACH ROW
DECLARE
    acc_balance NUMBER;
BEGIN
    SELECT Balance INTO acc_balance FROM Accounts WHERE AccountID = :NEW.AccountID;

    IF :NEW.TransactionType = 'Withdrawal' THEN
        IF :NEW.Amount > acc_balance THEN
            RAISE_APPLICATION_ERROR(-20001, 'Withdrawal amount is more than account balance.');
        END IF;
    ELSIF :NEW.TransactionType = 'Deposit' THEN
        IF :NEW.Amount <= 0 THEN
            RAISE_APPLICATION_ERROR(-20002, 'Deposit amount less than zero.');
        END IF;
    END IF;
END;
/
```

Output Evidence

Output screenshot is not available for this exercise.

SLF4J Logging Exercises

Exercise 1: Logging Error And Warning

Source Code

pom.xml - Libraries and Dependencies

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>com.example</groupId>
    <artifactId>LoggingExample</artifactId>
    <version>1.0-SNAPSHOT</version>

    <dependencies>
        <!-- SLF4J-->
        <dependency>
            <groupId>org.slf4j</groupId>
            <artifactId>slf4j-api</artifactId>
            <version>1.7.30</version>
        </dependency>

        <!-- Logback -->
        <dependency>
            <groupId>ch.qos.logback</groupId>
            <artifactId>logback-classic</artifactId>
            <version>1.2.3</version>
        </dependency>
    </dependencies>
</project>
```

LoggingExample.java

```
package com.example;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public class LoggingExample {
    private static final Logger logger = LoggerFactory.getLogger(LoggingExample.class);

    public static void main(String[] args) {
        logger.error("This is an ERROR Message");
        logger.warn("This is a WARNING Message");
    }
}
```

Output Evidence

Output Screenshot

Exercise 2: Parameterized Logging

Source Code

pom.xml - Libraries and Dependencies

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>com.example</groupId>
    <artifactId>ParameterizedLoggingExample</artifactId>
    <version>1.0-SNAPSHOT</version>

    <dependencies>
        <dependency>
            <groupId>org.slf4j</groupId>
            <artifactId>slf4j-api</artifactId>
            <version>1.7.30</version>
        </dependency>
        <dependency>
            <groupId>ch.qos.logback</groupId>
            <artifactId>logback-classic</artifactId>
            <version>1.2.3</version>
        </dependency>
    </dependencies>
</project>
```

ParameterizedLoggingExample.java

```
package com.example;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public class ParameterizedLoggingExample {
    private static final Logger logger =
        LoggerFactory.getLogger(ParameterizedLoggingExample.class);

    public static void main(String[] args) {
        String user = "Soumyadip Karan";
        int age = 20;

        logger.info("User {} has logged in", user);
        logger.debug("User {} is {} years old", user, age);
    }
}
```

Output Evidence

Output Screenshot

Exercise 3: Different Appenders

Source Code

pom.xml - Libraries and Dependencies

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>com.example</groupId>
    <artifactId>AppenderExample</artifactId>
    <version>1.0-SNAPSHOT</version>

    <dependencies>
```

```
        <dependency>
            <groupId>org.slf4j</groupId>
            <artifactId>slf4j-api</artifactId>
            <version>1.7.30</version>
        </dependency>
        <dependency>
            <groupId>ch.qos.logback</groupId>
            <artifactId>logback-classic</artifactId>
            <version>1.2.3</version>
        </dependency>
    </dependencies>
</project>
```

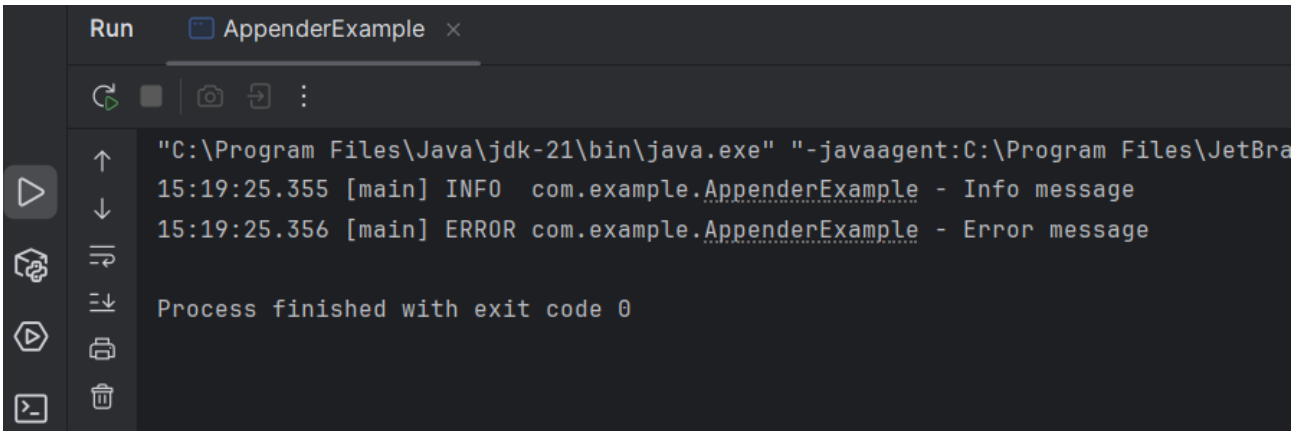
AppenderExample.java

```
package com.example;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public class AppenderExample {

    private static final Logger logger = LoggerFactory.getLogger(AppenderExample.class);

    public static void main(String[] args) {
        logger.info("Info message");
        logger.error("Error message");
    }
}
```

logback.xml - Logging Configuration

```
<configuration>

    <appender name="console" class="ch.qos.logback.core.ConsoleAppender">
        <encoder>
            <pattern>%d{HH:mm:ss.SSS} [%thread] %-5level %logger{36} - %msg%n</pattern>
        </encoder>
    </appender>

    <appender name="file" class="ch.qos.logback.core.FileAppender">
        <file>app.log</file>
        <encoder>
            <pattern>%d{HH:mm:ss.SSS} [%thread] %-5level %logger{36} - %msg%n</pattern>
        </encoder>
    </appender>

    <root level="debug">
        <appender-ref ref="console"/>
        <appender-ref ref="file"/>
    </root>

</configuration>
```

Output Evidence



Output Screenshot